

EchoVault Nexus

How It Works: an end-to-end guide

Personal Knowledge & Project Intelligence

EchoVault Nexus: How It Works

Personal Knowledge & Project Intelligence: a complete end-to-end guide

---

1. What EchoVault Nexus is

EchoVault Nexus is a **personal knowledge workspace**. It is a web application where you:

- **capture** what you learn as notes,
- **organise** those notes into projects,
- **record decisions** together with the reasoning behind them,
- **find** anything again with fast search, and
- **ask questions** about your own notes and decisions with an AI assistant that shows exactly which of your records it used.

It is built for developers, students, researchers, founders and anyone whose work depends on remembering *what they learned* and *why they decided something*.

---

2. The problem it solves

Knowledge workers lose time and make worse decisions because their knowledge is **scattered, disconnected and forgotten**:

Problem	What it looks like in real life
Scattered knowledge	Useful information lives in chat messages, bookmarks, random text files, emails and memory. When you need it, you can't find it.
Lost context	You remember *that* something was decided ("we use PostgreSQL") but not *why*, which alternatives were rejected, or whether it worked out. Months later the same debate starts again.
No connections	A note about rate limiting, the project it belongs to, and the decision it informed are stored in three unrelated places, so nothing links them together.
Untrustworthy AI answers	General AI chatbots answer confidently from the internet, not from *your* knowledge, and you cannot check where an answer came from. Pasting private notes into public chatbots is also a privacy risk.

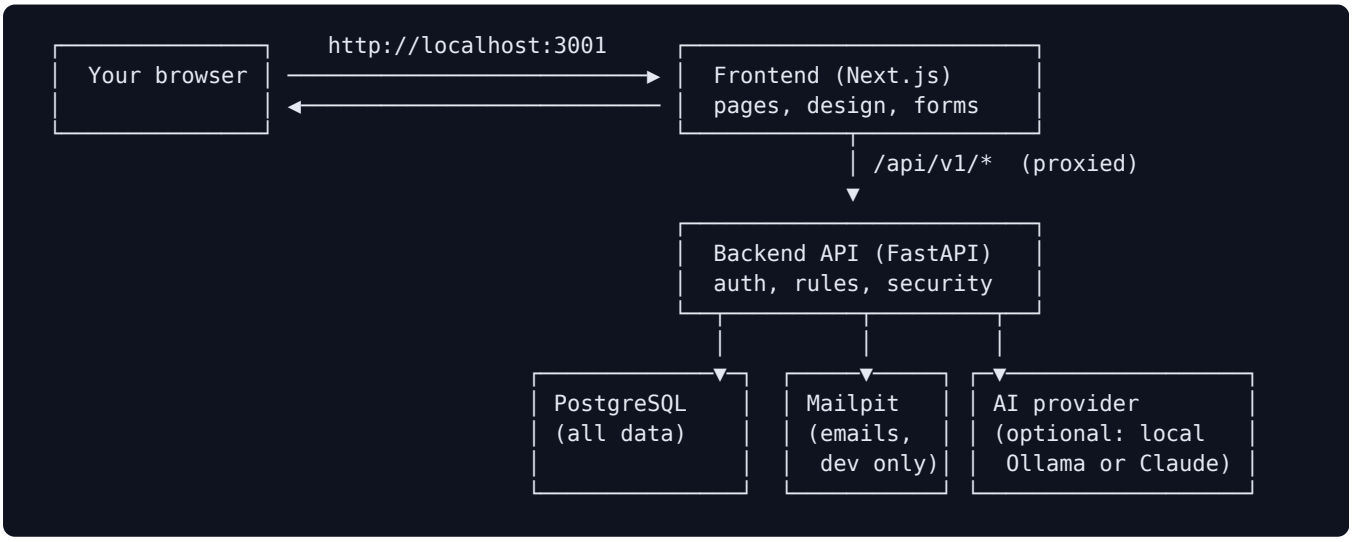
---

3. The solution

EchoVault Nexus brings these pieces into **one private, connected workspace**:

Need	How EchoVault Nexus answers it
Capture quickly	**Knowledge Vault**: Markdown notes with tags, project links and related-note links.
Keep work organised	**Projects**: group notes and decisions; track status (active, on hold, completed, archived).
Remember *why*	**Decision Journal**: context, problem, alternatives, selected option, reasoning, expected vs actual outcome, status and review date.
See connections	**Connection maps**: each note shows its project, that project's decisions, and related notes.
Find anything	**Global search**: PostgreSQL full-text search with partial-word matching, filters and highlighted results.
Ask your own knowledge	**Source-grounded AI assistant**: answers *only* from your notes and decisions, links every source, and says plainly when your notes don't cover a question.
Stay private	Every record belongs to one account; the server checks ownership on every request. Passwords are hashed with Argon2. AI use is optional and only sends your own relevant records.

4. How the system is built (the big picture)



Part	Technology	Job
Frontend	Next.js 16 (Pages Router), React 19, TypeScript, Tailwind CSS 4, DaisyUI 5	Everything you see and click. Runs on port **3001** in development.
Backend API	Python, FastAPI, SQLModel, Pydantic	Accounts, sessions, business rules, search, the assistant. Runs on port **8010**.
Database	PostgreSQL 17 (Docker), Alembic migrations	Stores users, notes, projects, decisions, tags, links and sessions. Port **5433**.
Email tester	Mailpit (Docker)	Catches emails such as password resets in development. Inbox at **http://localhost:8025**.
AI provider	Ollama (free, local) or Anthropic Claude	Generates assistant answers from your retrieved records. Optional.

Why the frontend "proxies" the API: the browser only ever talks to `localhost:3001`. Next.js forwards anything under `/api/v1/` to the backend. This keeps the login cookies *first-party* (same site), which is safer and avoids cross-site cookie problems.

---

5. The running example

To make every step concrete, this guide follows one user:

> **Ada** is a backend developer. Her team is launching a **public API**. She needs to remember how rate limiting works, record *why* the team chose a particular approach, and later ask "what rate limit did we choose and why?" without digging through old chats.

---

6. End-to-end walkthrough

Step 0: Starting the application

From the project folder, one command starts everything:

```
cd ~/Documents/CAAPSTOON-APF/Echovault
./dev.sh
```

What `dev.sh` does, in order:

1. Checks prerequisites (Python environment, `.env` file, installed frontend packages, free ports).
2. Starts **PostgreSQL** and **Mailpit** in Docker.

3. Applies any pending **database migrations** (Alembic), so the tables match the code.
4. Starts the **backend** on `http://127.0.0.1:8010` and the **frontend** on `http://localhost:3001`.
5. Prints `✓ EchoVault Nexus is starting. Open http://localhost:3001`.

Pressing **Ctrl+C** stops both servers. The terminal must stay open while you use the app.

Step 1: The landing page (`/`)

Ada opens `http://localhost:3001`. She sees:

- The headline **"Your knowledge, connected to every decision."**
- An animated **knowledge graph**: lines draw themselves from a note to its project, a decision and two references, showing what the product does. It plays once; users who prefer reduced motion see the finished graph immediately. On phones it becomes a readable vertical chain.
- Sections for the five features, a **sample decision record**, and a **Privacy & security** section that lists only protections that are actually implemented.
- A footer linking to the **Terms, Privacy Policy, Acceptable Use, Cookie Policy and Contact** pages.

Illustrations are clearly labelled **"Illustration with sample data"**: nothing on the landing page pretends to be real user data.

Step 2: Creating an account (`/register`)

Ada clicks **Create your vault** and fills in:

Field	Ada enters	Rule
Full name	Ada Lovelace	required, up to 120 characters
Email	ada@example.com	must be a valid email; stored in lower case
Password	`rate limits are fun`	10–128 characters (length matters more than symbols)
Terms checkbox	ticks it	**starts unticked**; must be ticked to continue

Below the checkbox a separate line explains that her data is handled as described in the Privacy Policy. There is **no marketing consent** bundled in.

What happens behind the scenes when she clicks "Create your vault":

1. **Browser check:** the form validates instantly (e.g. "Use at least 10 characters").
2. **Request:** the browser sends `POST /api/v1/auth/register` with her details, `accept_terms: true` and the **Terms version** the page displayed.
3. **Server validation:** the backend re-checks everything independently, because a browser can never be trusted. If the Terms version is outdated, it asks her to reload and review the new Terms.

4. **Uniqueness:** the email must not already exist (the database enforces this even if two sign-ups happen at the same instant).
5. **Password hashing:** the password is turned into an **Argon2 hash**. The real password is never stored or logged.
6. **Consent record:** the accepted Terms version and the time of acceptance are saved with the account.
7. **Session created:** the server creates a login session and sets three cookies (see Step 3).
8. **Redirect:** Ada lands on her **Overview**.

Rate limit: at most 10 registrations per hour from one network address, to slow down automated abuse.

Step 3: Signing in and staying signed in (`/login`)

Next time, Ada signs in with her email and password.

- A wrong password or an unknown email both return the **same message**: "Invalid email or password". This prevents attackers from discovering which emails have accounts.
- Sign-in is **rate-limited**: 8 attempts per email and 30 per network address every 15 minutes.
- If she was trying to open a protected page (e.g. `/vault`), she is returned there after signing in.

How the session works (in plain language):

Cookie	What it is	Lifetime	Readable by page scripts?
`ev_access`	A short-lived signed pass (JWT) proving she is signed in	15 minutes	No (HttpOnly)
`ev_refresh`	A long-lived renewal token, only sent to sign-in endpoints	14 days	No (HttpOnly)
`ev_csrf`	A random value used to block forged requests from other websites	14 days	Yes, by design

1. Every request carries the access pass. The server also checks that the **session has not been revoked**, so signing out takes effect immediately.
2. When the 15-minute pass expires, the app quietly asks for a new one using the refresh token. The refresh token is **rotated** (replaced) each time; if an old one is ever reused, the server assumes it was stolen and ends the session.
3. Every change (save, delete, etc.) must include the CSRF value in a header. Another website cannot read that value, so it cannot forge changes on Ada's behalf.
4. **Sign out** revokes the session on the server and clears the cookies. **Sign out everywhere** (in Settings) ends every session on every device.

Step 4: The Overview (`/dashboard`)

After signing in, Ada sees **"Good morning, Ada"** and:

- **Real counts:** notes (with archived count), active projects and decisions. The numbers count up briefly when the page loads.
- **Recent activity:** her latest notes, projects and decisions mixed together, newest first, each labelled with its type.
- **Active projects**, **recently updated notes** and **recent decisions**.
- **Getting started** steps while her account is empty.
- A search bar (press **/** anywhere to jump to search).

All numbers come from the database: nothing is hard-coded.

Step 5: Creating a project (`/projects/new`)

Ada creates a project:

Field	Value
Name	Public API launch
Description	Ship v1 of the public REST API with keys, docs and limits.
Status	Active

The project page then shows its description, a **status selector** (active, on hold, completed, archived), and lists of **linked notes** and **linked decisions**, each with an **Add** button that pre-selects this project.

Deleting a project keeps its notes and decisions. They simply become "not in a project". The confirmation dialog says exactly this before anything happens.

Step 6: Capturing knowledge in the Vault (`/vault/new`)

From the project page Ada clicks **Add** under Knowledge. The project is already selected. She writes:

Title: Token bucket rate limiting

Content (Markdown):

```
## How it works
Each API key gets a **bucket** that refills at a fixed rate.

- Refill: 100 requests per minute
- Burst: 20
- Over the limit: respond with HTTP 429
```

Tags: `api`, `performance` · **Related notes:** she searches "nginx" and links her existing note *Nginx rate limit config*.

Editor features:

- **Write / Preview** tabs: Markdown is rendered safely (no raw HTML is ever executed).
- **Save status:** "Unsaved changes", "Saving…", or "No unsaved changes".
- **Unsaved-changes warning** if she tries to leave mid-edit.

What the database stores: the note (title, content, owner, project, timestamps), its tags (re-usable per user), and one **two-way link** to the related note. Link targets are checked: she can only link to her *own* notes.

The note page (`/vault/<id>`) shows the rendered content; a side panel with project, related notes, tags and dates; **Edit**, **Archive** and **Delete** buttons; and the **Connections** map (Step 8).

Vault list (`/vault`): search box, tag filter, project filter, sort order, **Active / Archived** tabs and pagination. Filters are kept in the address bar, so a filtered view can be refreshed or bookmarked.

Step 7: Recording a decision (`/decisions/new`)

The team decides how to rate-limit. Ada records it:

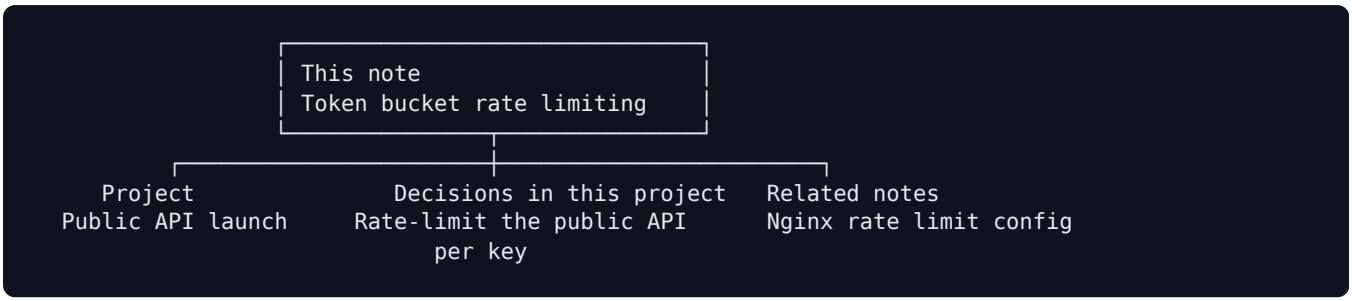
Field	Ada's entry
Title	Rate-limit the public API per key
Status	Accepted
Project	Public API launch
Context	A single client can exhaust the database connection pool.
Problem statement	Stop abusive bursts without blocking legitimate integrations.
Alternatives considered	Per-IP limits at the proxy; scale the database instead.
Selected option	Token bucket per API key, 100 requests/minute with bursts of 20.
Reasoning	Fair per customer, and needs no extra infrastructure.
Expected outcome	No more pool exhaustion; under 1% of requests limited.

The **decision page** puts the **selected option and reasoning first**, then context and problem, alternatives, and **expected vs actual outcome side by side**.

Six weeks later Ada clicks **Add review notes** and writes: "Zero pool incidents; 0.3% limited, mostly one buggy client." The app records a **last-reviewed date** automatically. She can also click **Mark reviewed** to change the status to *Reviewed*. Statuses: **Proposed**, **Accepted**, **Superseded**, **Reviewed**, always shown with an icon and text, never colour alone.

Step 8: Seeing connections

On the note *Token bucket rate limiting*, the **Connections** map shows:



Every item is a real record and a clickable link. The connector lines grow into place when the page opens.

Step 9: Finding things with search (`/search`)

Ada presses **/**, types `limit` and presses Enter.

- Results are grouped into **Notes**, **Projects** and **Decisions**, with counts on filter tabs (All, Notes, Projects, Decisions).
- **Partial words match**: `limit` finds "limiting" and "limits"; `deploy` finds "deployment".
- Matching words are **highlighted** in titles and previews.
- Tags match too: searching `perf` finds notes tagged `performance`.
- Each result shows a status label (e.g. *Accepted*, *Archived*).
- A single-type tab supports **pagination** for large result sets.

Behind the scenes: PostgreSQL full-text search with special indexes (GIN) makes this fast, and every query is restricted to **Ada's own records**.

Step 10: Asking the AI assistant (`/assistant`)

Ada asks: **"What rate limit does our public API use, and why?"**

What happens, step by step:

1. **Retrieval:** the backend searches *only Ada's* records for matching words, across **notes and decisions**. Archived notes are skipped. The best matches (up to 6) are chosen.
2. **No match? No AI call.** If nothing relevant is found, Ada is told so immediately and **nothing is sent to the AI provider**.
3. **Safe packaging:** each record is wrapped as a numbered `<source>` and treated as *untrusted data*. Instructions hidden inside a note (for example "ignore your rules") are not followed.
4. **Answer:** the AI provider writes an answer using only those sources, citing them like `[1]`, `[2]`.
5. **Verifiable sources:** the page lists every source used, each a link to the original note or decision, marked *cited* or *retrieved but not cited*.

Example (verified with the free local model):

> **Answer:** "Our public API uses a token bucket per API key: 100 requests per minute with a burst of 20. Requests over the limit receive HTTP 429. The approach was chosen because it is fair per customer and needs no extra infrastructure."
>
> **Sources:** Token bucket rate limiting (note) · Rate-limit the public API per key (decision)

If Ada asks "What is the capital of Mongolia?", she sees: *"I couldn't find anything in your notes or decisions that relates to this question, so I haven't sent it to the AI provider."*

AI providers:

Option	Cost	Speed on a laptop	Notes
Ollama with `qwen2.5:1.5b` (current setup)	Free	about 15–40 s	Runs on your computer; notes never leave it. Small model: may not write `[1]` markers.
Anthropic Claude	Paid per use	a few seconds	Best quality and citations. Needs an API key with credit.

If no provider is configured, the assistant page says so honestly. It never shows fake answers.

Step 11: Tidying up: archive, restore, delete

- **Archive** a note to hide it from the vault list and the assistant while keeping it searchable. **Restore** it any time from the *Archived* tab.
- **Delete** a note, project or decision through a confirmation dialog. Deleting a project keeps its notes and decisions.

Step 12: Managing the account

Where	What Ada can do	What happens
Profile	Change her name	Saved immediately.
Profile	Change her sign-in **email**	Requires her current password. The **old address** receives a warning email (without revealing the new one).
Profile	Change her **password**	Requires the current password; signs her out on every *other* device.
Sign-in page	**Forgot password**	She receives an email with a single-use link that expires in 30 minutes. In development the email appears in **Mailpit** at `http://localhost:8025`. The same response is shown whether or not the email exists.
Settings	Switch theme (Dark, Light, System)	Remembered in her browser.
Settings	**Download my data**	A JSON file with her account, notes (including archived), tags, links, projects and decisions.
Settings	See which Terms version she accepted	Shown with the date.
Settings	**Sign out everywhere**	Ends all sessions on all devices.
Settings	**Delete account**	Requires password and typing `DELETE`; permanently removes the account and all its data.
Any page	**Updated Terms** prompt	If the Terms change, she must review and accept them (unticked box) to continue; Settings stays usable so she can export or delete instead.

---

7. Security and privacy, summarised

Protection	How it is implemented
Passwords	Argon2 hashing; never stored or logged in plain text.
Sessions	HttpOnly cookies; 15-minute access passes; rotating refresh tokens; server-side revocation.
Forged requests (CSRF)	Every change requires a secret header that other websites cannot read; cookies are `SameSite=Lax`.
Account isolation	Every database query filters by the signed-in user. Another user's record returns **"Not found"**, even with the right ID. IDs are random UUIDs.
Abuse	Rate limits on sign-in, registration, password reset, email change, export, deletion and the assistant.
Safe content	Markdown is rendered without raw HTML; search highlighting never injects HTML.
AI privacy	Only the user's own, relevant, non-archived records are sent, and only when they ask; nothing is sent when nothing matches.
Errors	Users see clear messages; technical details go only to server logs (which exclude passwords, tokens and note content).
Legal	Terms, Privacy, Acceptable Use, Cookies and Contact pages describe the real implementation; Terms acceptance is recorded with a version and time.

8. What is stored (data model)

Table	Holds
`users`	Name, email, Argon2 password hash, accepted Terms version and time
`user_sessions`	One row per signed-in device: hashed refresh token, expiry, revoked time
`password_reset_tokens`	Hashed single-use reset tokens with expiry
`projects`	Name, description, status, owner
`knowledge_items`	Notes: title, Markdown content, project, archived time, owner
`tags` + `knowledge_item_tags`	Per-user tags and which notes use them
`knowledge_links`	Two-way "related note" links
`decisions`	Context, problem, alternatives, selected option, reasoning, outcomes, status, review date, project, owner

Every table that holds user content has an **owner**, and deleting an account removes everything that belongs to it.

---

9. How quality is checked

Check	What it covers
Backend tests (pytest, 69)	Sign-up rules, sessions, CSRF, rate limits, CRUD for everything, archive, related links, decision status, search, export, email change, account deletion, assistant behaviour and **cross-user isolation**.
Frontend unit tests (Vitest, 23)	API client (CSRF, refresh, errors), forms, Terms prompt, email links, highlighting.
Browser tests (Playwright, 23)	Real user journeys in Chromium, including password reset through a real email, and a **no-horizontal-overflow check on 19 pages at 7 screen widths (320–1440 px)**.
Continuous integration	Every change on GitHub runs all of the above automatically.

---

10. Current limitations and next steps

- **Search is keyword-based** (full-text), not "meaning-based"; semantic search with pgvector is a planned enhancement.
- The free local AI model is slower and less precise than Claude; Claude needs a funded API key.
- A new email address is not confirmed by a link; the change takes effect after the password check.
- Legal pages contain **[placeholders]** for business details (name, address, contact emails, jurisdiction) and need a lawyer's review before a public launch.
- The app has **not been deployed** to the internet yet; the README contains a deployment guide.

---

11. Glossary

Term	Meaning
API	The set of web addresses the frontend calls to read and change data.
Argon2	A modern, deliberately slow password-hashing method that makes stolen password databases hard to crack.
CSRF	Cross-Site Request Forgery: a trick where another website makes your browser perform actions on a site you're signed into.
Full-text search	Searching the words inside documents, with word stems and ranking, rather than exact matches only.
HttpOnly cookie	A cookie page scripts cannot read, which protects session tokens from theft.
JWT	A signed token proving identity for a short time.
Migration	A versioned script that changes the database structure safely.
Source-grounded AI	An AI answer built only from supplied documents, with references back to them.

EchoVault Nexus: built with Next.js, FastAPI and PostgreSQL. For setup and deployment details, see the project README.